

Classroom Habits

- Place bags under chairs so that it is easy to walk around.
- Our goal is to learn! And the first step is not being distracted!
 - Phones
 - Place your phone on Do Not Disturb so that it does not distract from your learning.
 - Phones should be placed in your bags during class.
 - Laptops
 - Should only be out if you are using them to view the slides
- We are so fortunate to be learning in this room. At the end of class please
 - Clean white boards
 - Place white boards back where they belong
 - Tuck in your chair
 - Clean up trash

A decorative graphic on the left side of the slide, consisting of a network of thin, gold-colored lines and small circles, resembling a circuit board or a stylized tree structure.

Unix Basics

COMP211 Connor McMahon

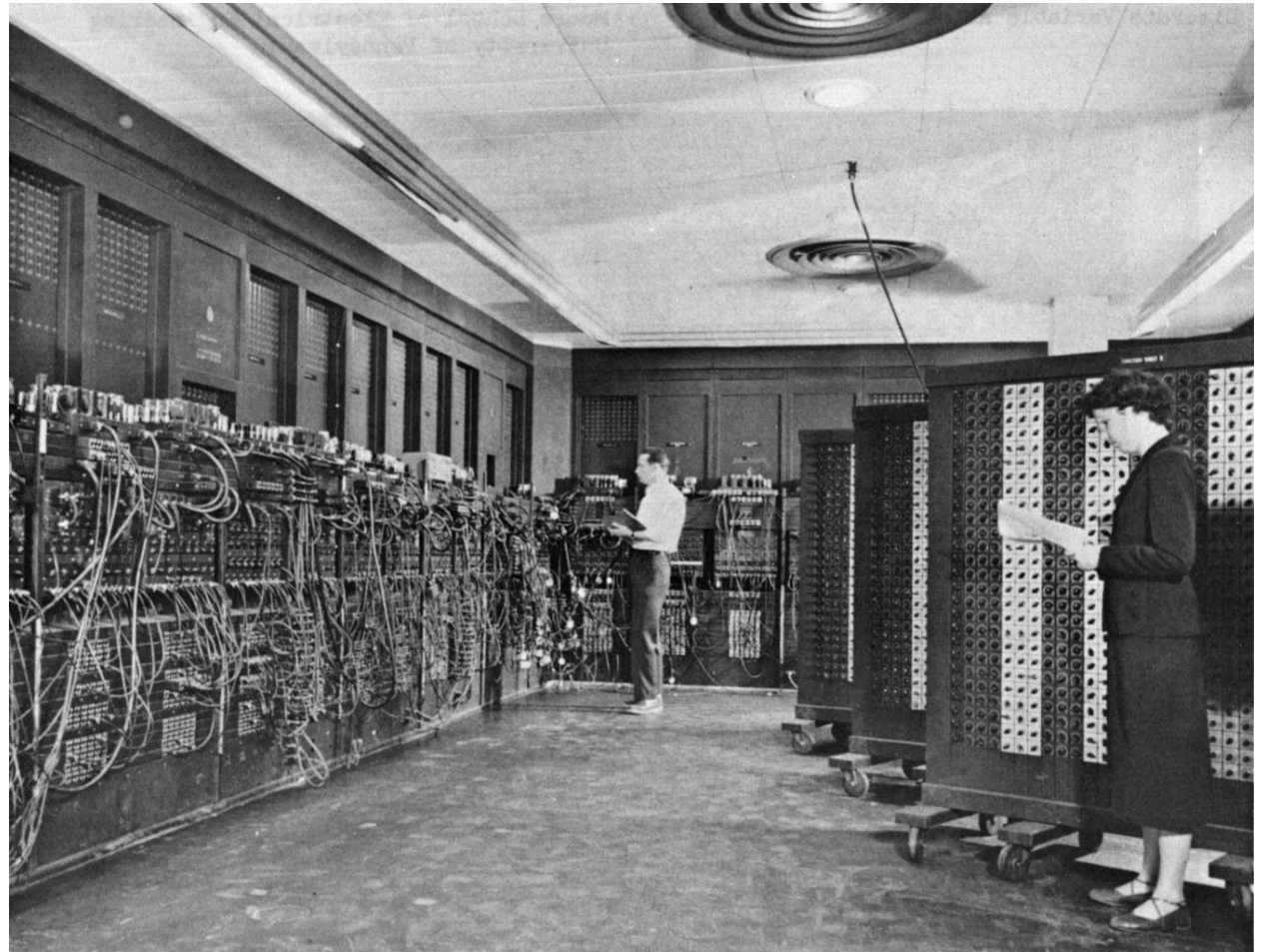
Terminal

Terminal

- In the past, anyone who interacted with a computer did so through a **terminal**
 - That is no longer true today
- Now, most users don't use the terminal (or even know about it!)
 - Instead, they interact through graphical interfaces
- But you are not most users! You are computer scientists!
- The terminal will be an essential tool in your toolkit

Early Computers

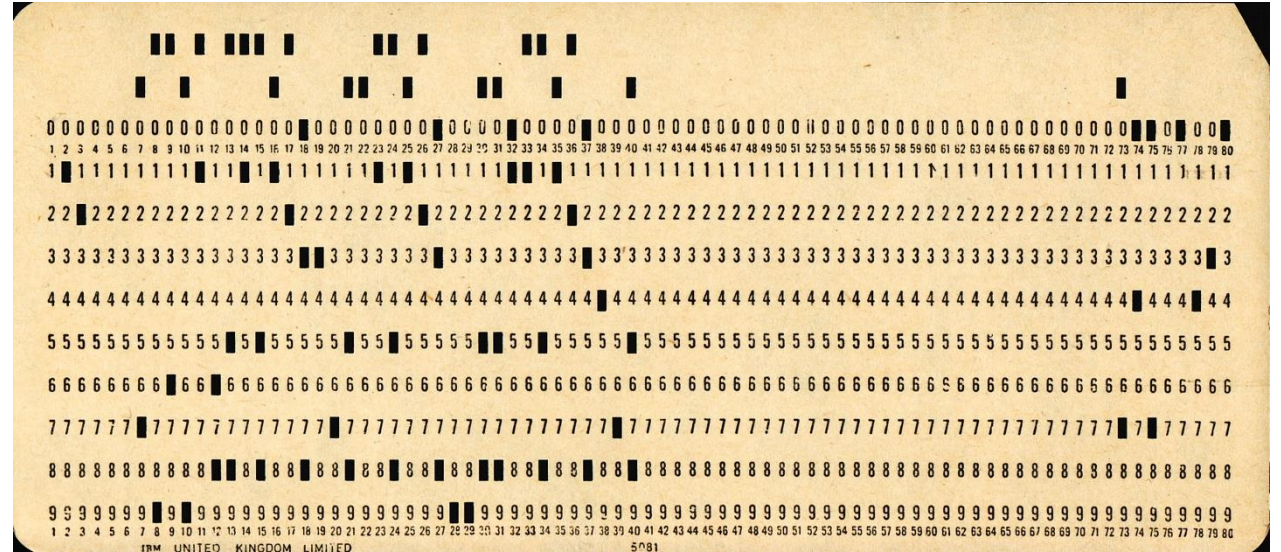
- The first electronic computers (like **ENIAC**, EDSAC, and early IBM machines) had no concept of interaction.
- They had no screens, mice, or graphics. They were room-sized machines that processed data in batches.



Glenn A. Beck (background) and Betty Snyder (foreground) program ENIAC

Early Computers: How they worked

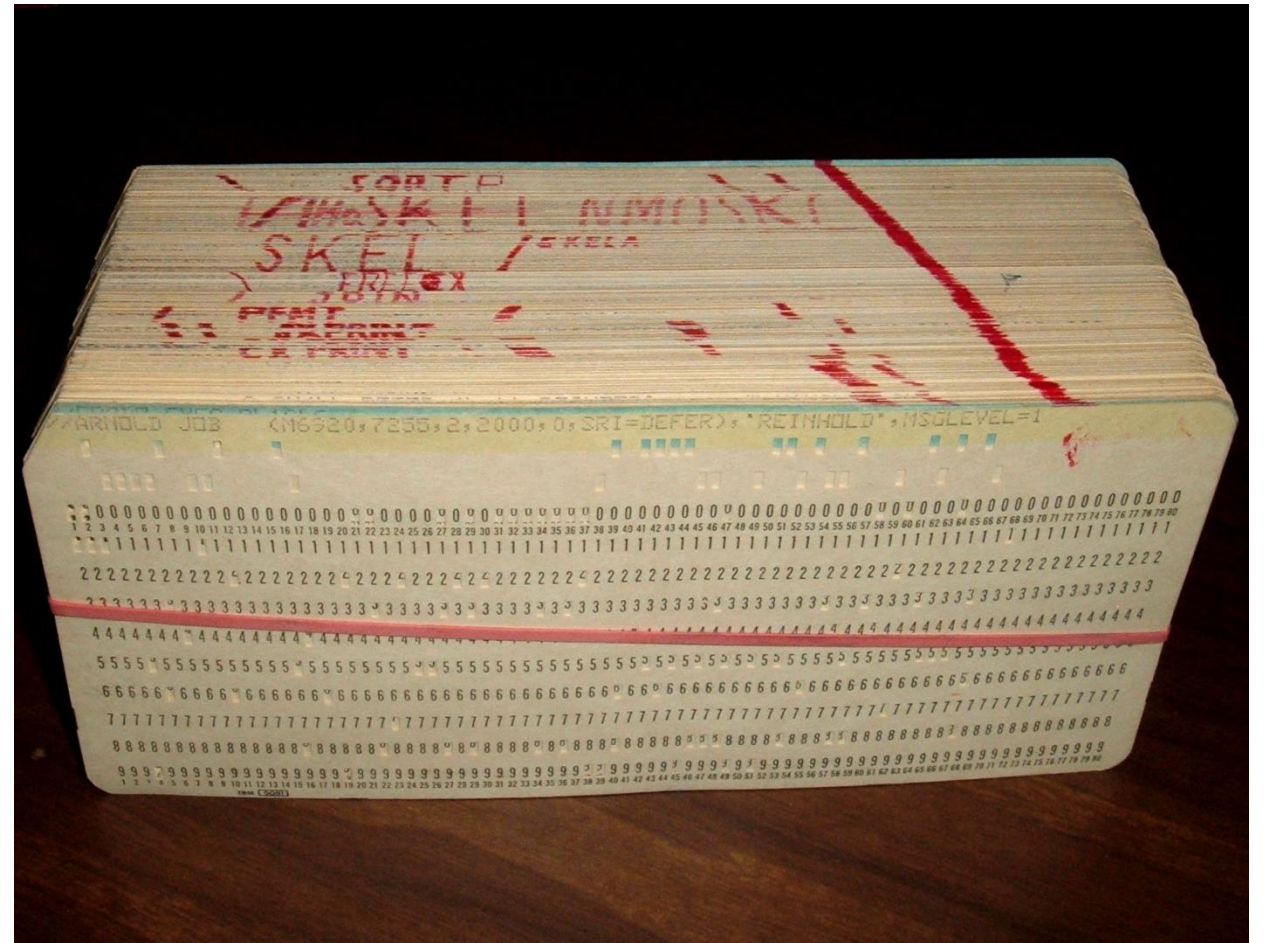
- **Batch processing**
- Input was encoded plugboard cables, switches, or punched cards.
- The computer ran the job.
- Output was printed hours, or days, after.
- There was no human interaction with the computer.



A 12-row/80-column IBM punched card from the mid-twentieth century. Remained common into the 1970s and early 1980s.

Early Computers

- There was no human interaction with the computer.
 - You couldn't “try something and see what happens”
- Debugging was painful: one mistake meant re-running the entire job



A deck of punched cards comprising a computer program. The red diagonal line is a visual aid to keep the deck sorted.

Real-time input & output

- Introducing: **Teletypes (TTYs)**
 - Originally designed for telegraph networks
- Engineers created the first **interactive text interface** using teletypes.
 - A human would use the keyboard to type characters to send to a computer
 - The computer would send characters back and they would be printed out line by line on paper.
- Teletypes connected to computers were often called:
 - Console **terminals**
 - Teletype **terminals**



Teletype Model 33 ASR teleprinter, with punched tape reader and punch, usable as a computer terminal.

Shift Towards Visual Displays

- Cathode-ray tubes (CRTs) were already used in:
 - Television
 - Radar
 - Oscilloscopes



Typical 1950s United States monochrome CRT TV

Cathode-Ray Tubes

- Engineers integrated CRTs into computers because they allowed
 - Cursor movement
 - Overwriting text
 - Screen-based editing
 - Faster feedback
- As CRT-based devices appeared, the word **terminal** stuck, even though the technology changed.



VT100

In the beginning, there was only text, no images

- Hardware limitations
 - Early computers had very little memory
 - Graphics require far more data than text
 - Text is compact, fast, and simple.

In the beginning, there was only text, no images

- Hardware limitations
 - Early computers had very little memory
 - Graphics require far more data than text
 - Text is compact, fast, and simple.
- Computers spoke “commands,” not pictures
 - Computers fundamentally operate on symbolic instructions:
 - LOAD
 - ADD
 - JUMP
 - PRINT
 - Typing commands mapped naturally to how machines worked internally.

A philosophical shift: computers as interactive partners

Before

Computers = machines
that run jobs

After

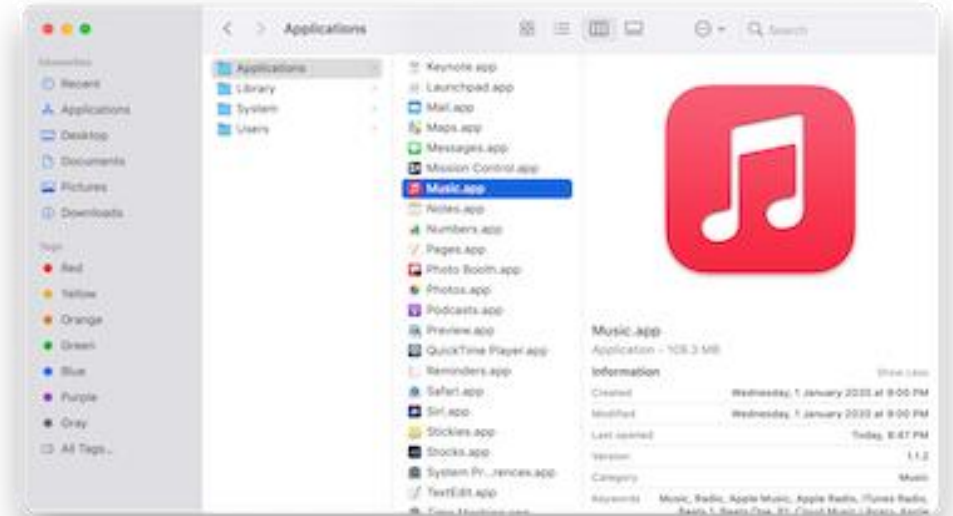
Computers = systems
you work with

A philosophical shift: computers as interactive partners

- This enabled:
 - Interactive programming
 - Real-time debugging
 - Exploratory computing
- It also laid the groundwork for:
 - Networking
 - Remote login
 - Eventually, personal computers

The shift away from “terminal-only”

- In the late 1970s–1980s:
 - Cheaper memory
 - Better, more affordable displays
 - The invention of the mouse
- This led to:
 - Graphical User Interfaces (GUIs)
 - Windows, icons, menus
 - Computers becoming accessible to non-experts



The terminal is still here!

- Even today:
 - Servers often have no GUI at all
 - Programmers, system administrators, and researchers rely on it daily
- That's because the terminal is:
 - Scriptable
 - Remote-friendly
 - Extremely powerful
- Every computer scientist should know how to work with the terminal

How Terminal Interaction Works

- A terminal session follows a simple loop:
 - Prompt appears
 - User types a command
 - Presses Enter
 - Computer responds with text
 - Prompt appears again
- This style is called a **command-line interface (CLI)**.

Command-line Interface (CLI)

- A **CLI** is a way of interacting with a computer by typing commands instead of clicking buttons.
 - It's the interface style, not a specific program.
- Examples:
 - Bash CLI
 - PowerShell CLI
 - Git's command-line usage

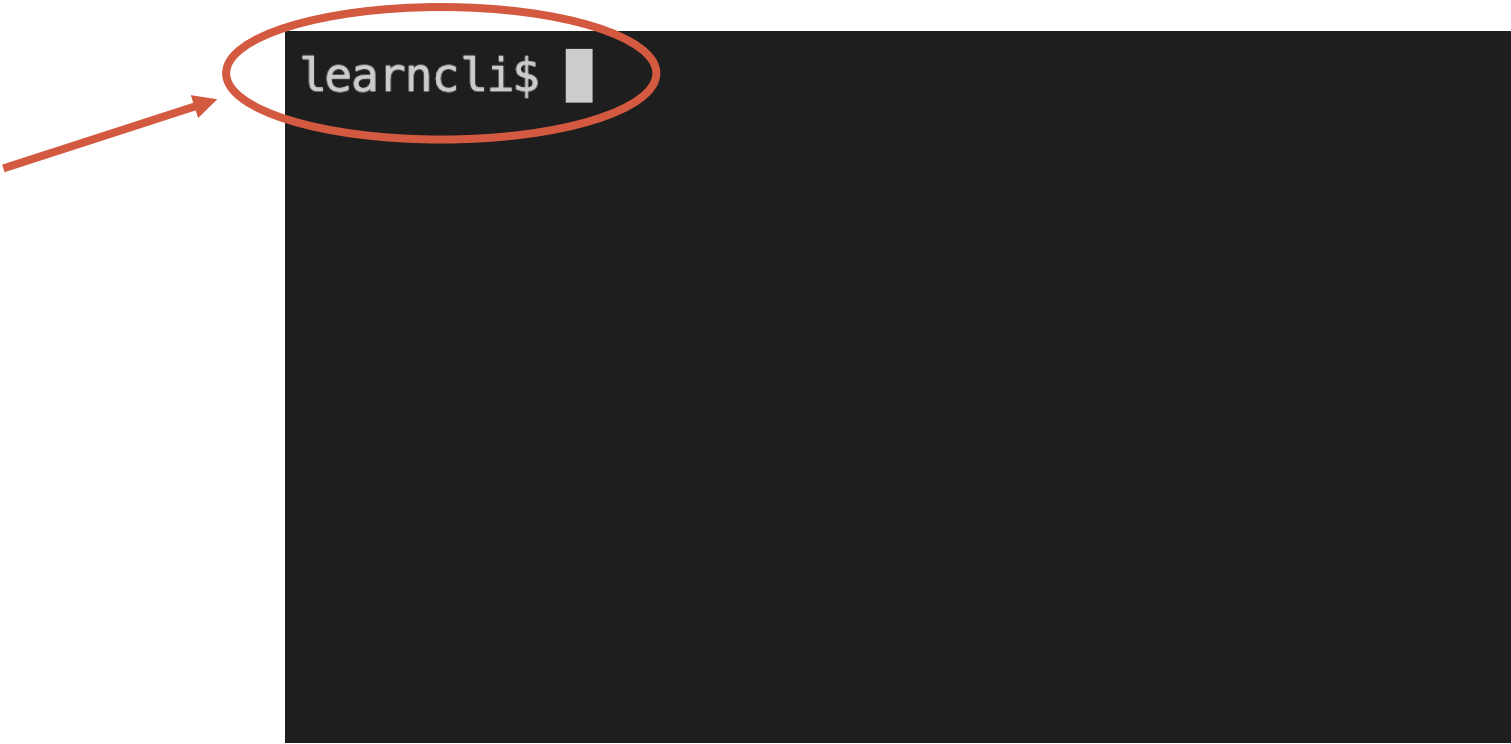
Shell

- A **shell** is the program that actually interprets and executes your commands.
 - It's the command interpreter.
- Common shells:
 - bash
 - zsh
 - fish
 - sh
 - PowerShell
 - cmd.exe

Bash

- The shell we will use in this class is called **Bash**
- It is the default shell on Linux
- When you open a terminal for this class, it will auto-launch a bash shell:

This is the shell prompt. It's the shell's way of saying, "I'm ready for you to give me a command."



```
learncli$ █
```

Command-line Interface

A text-based way to interact with a system

Terminal

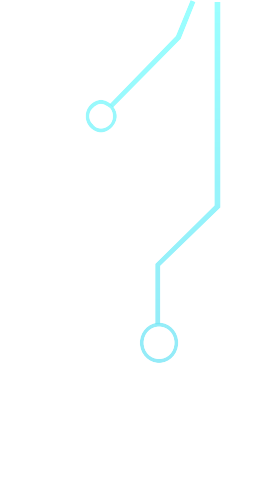
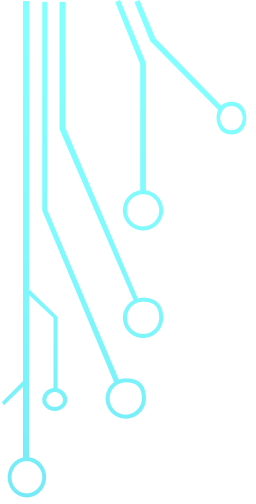
The app/window you type into

Shell

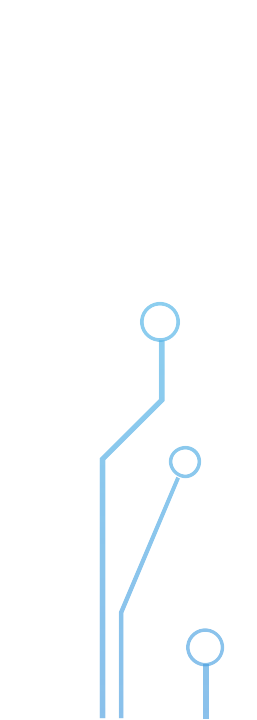
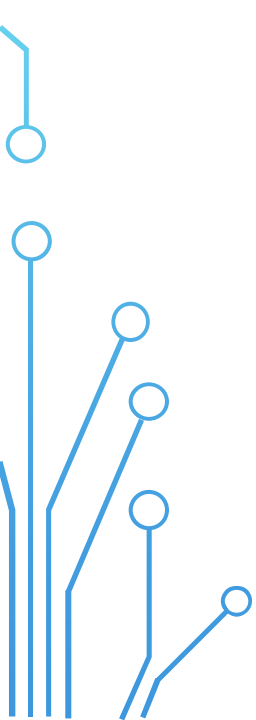
The program that understands and runs your commands

Terminal

- A **terminal** is an application that displays text input/output and lets you interact with a shell.
 - It's the window or app where you type commands.
- Examples:
 - Terminal.app (macOS)
 - Windows Terminal
 - GNOME Terminal
 - iTerm2
- What it does:
 - Shows text
 - Sends your keystrokes to a shell
 - Displays the shell's output



Unix File System



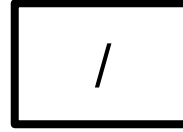
Unix

- **Unix** is a family of operating systems.
- The **operating system** is the “boss” of your computer.
 - It manages memory, files, programs, and hardware.
- Many modern systems are based on or inspired by Unix.
 - Eg. macOS is a Unix operating system

File System

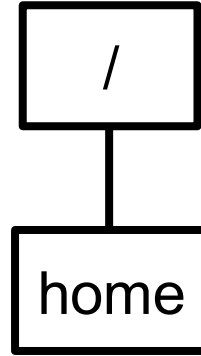
- **File system:** the way an operating system organizes and stores files.
- Most operating systems use a **hierarchical** (tree-like) structure.
- **Root**
 - At the top of the tree is the **root** directory.
 - On Mac/Linux, this is **/**
 - On Windows this is a drive letter like **C:**
- **Directories** (Folders)
 - Directories act like branches in the tree.
 - They can contain files or other directories (subdirectories).
- **Files**
 - Files are the leaves of the tree.
 - A file stores data (text, images, programs, etc.).

Unix File System



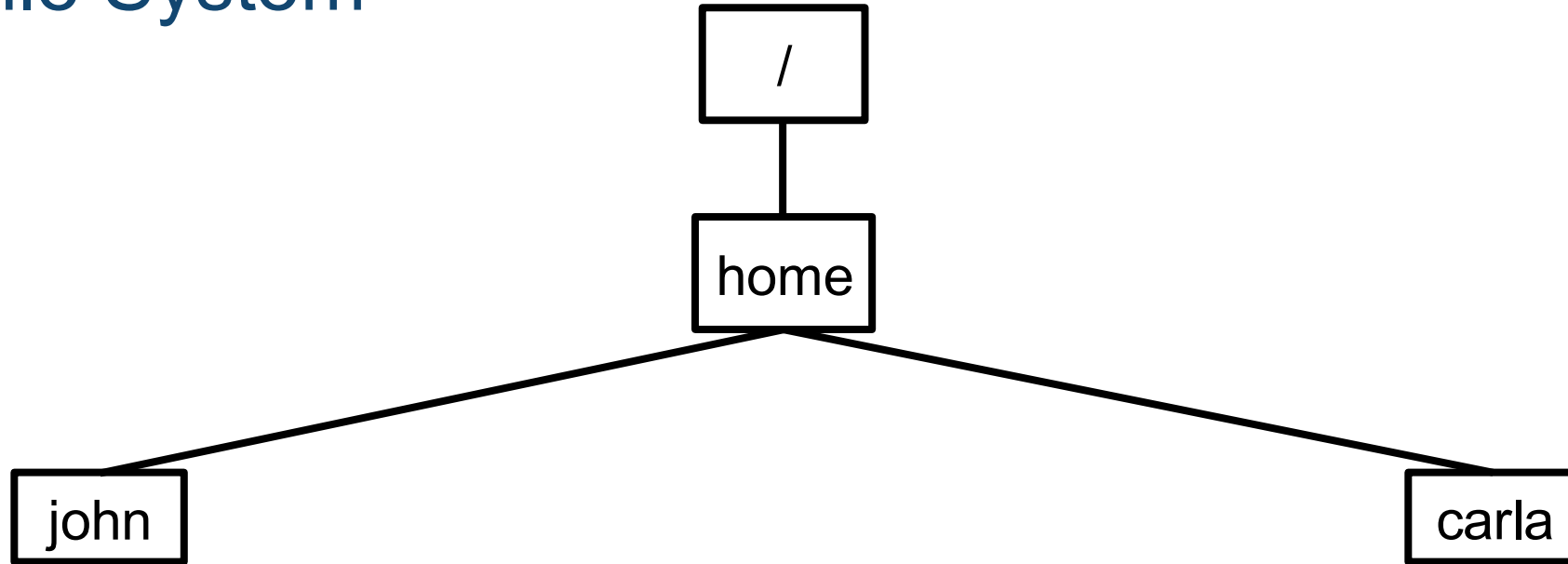
The root directory is at the top of the tree.

Unix File System



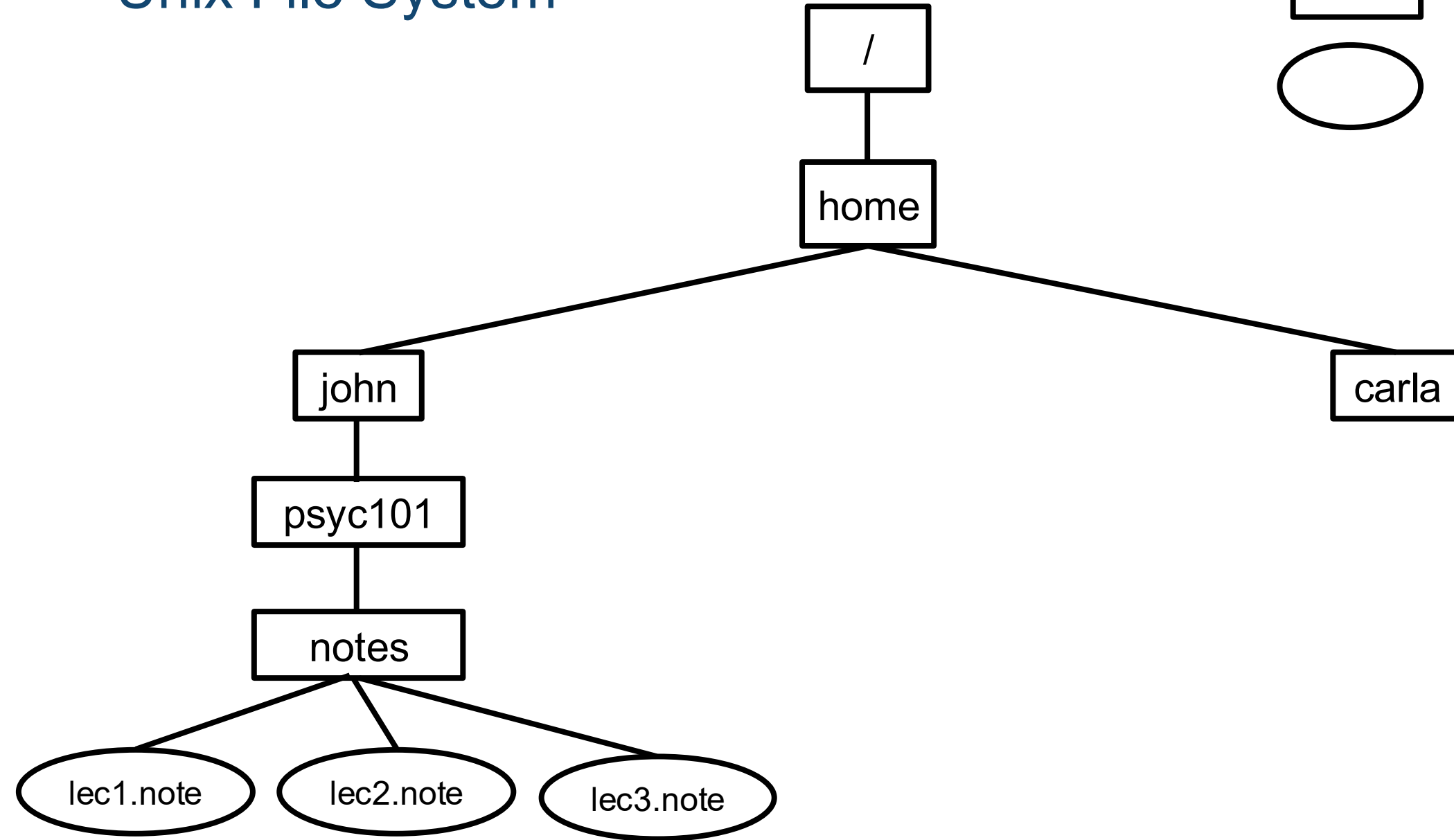
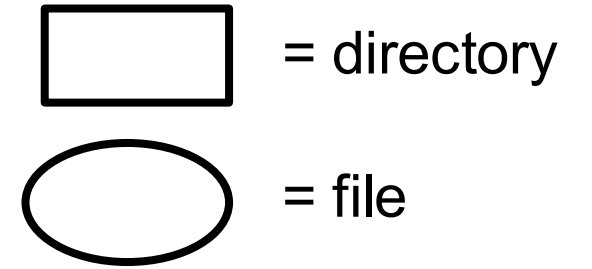
home stores each user's home directory which has the same name as the usernames.

Unix File System

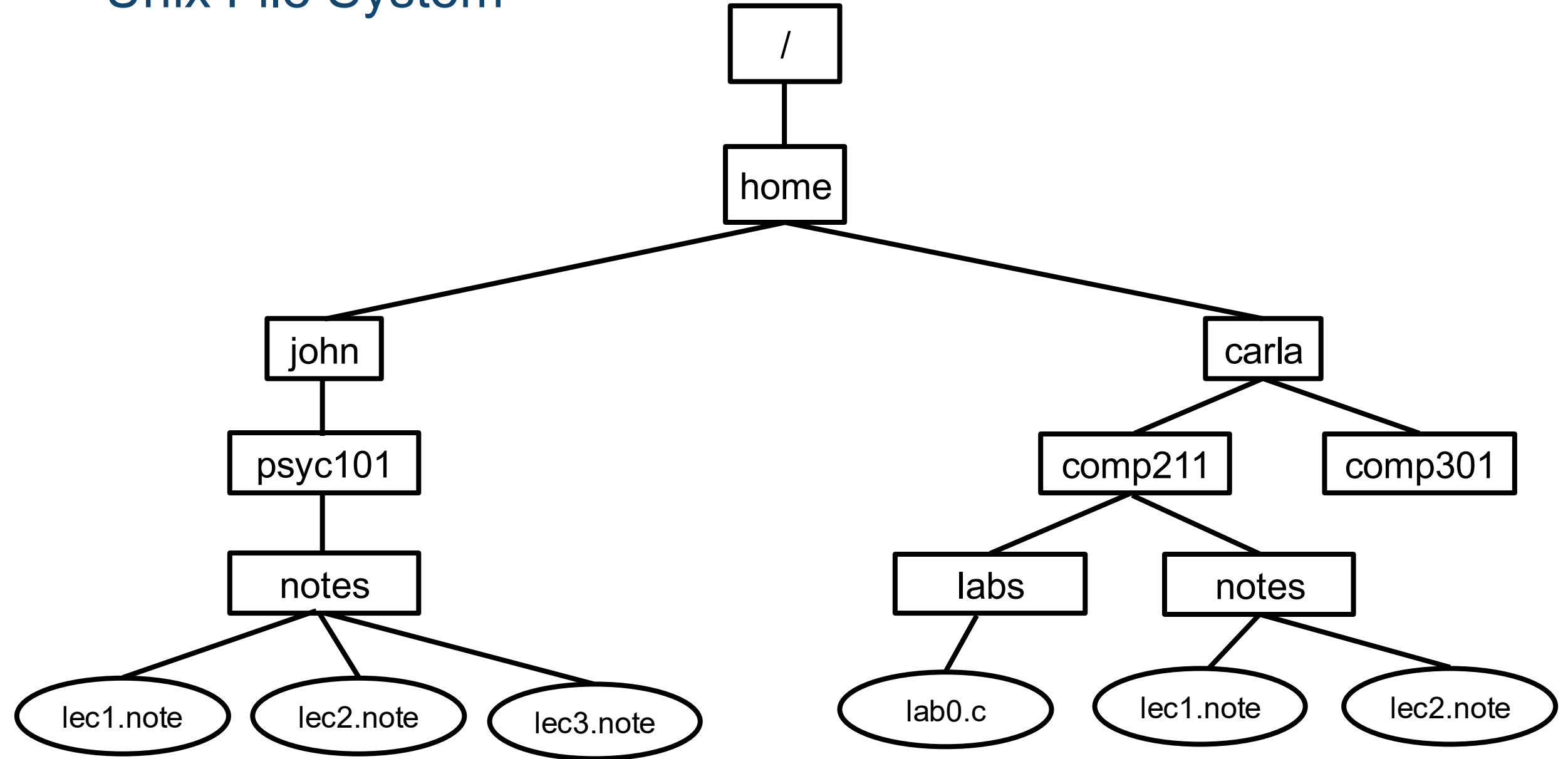


Let's say there are two users – john and carla

Unix File System

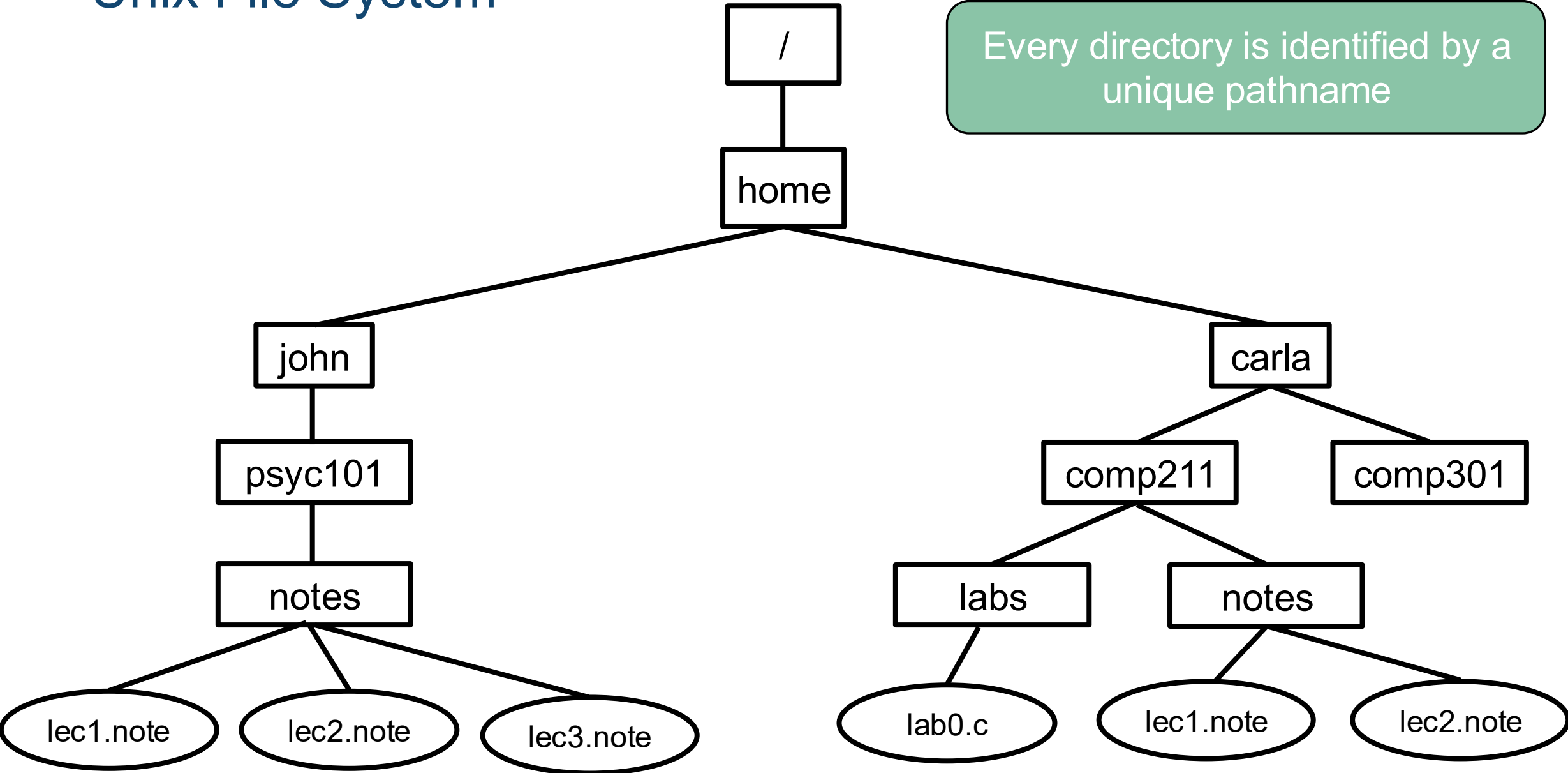


Unix File System

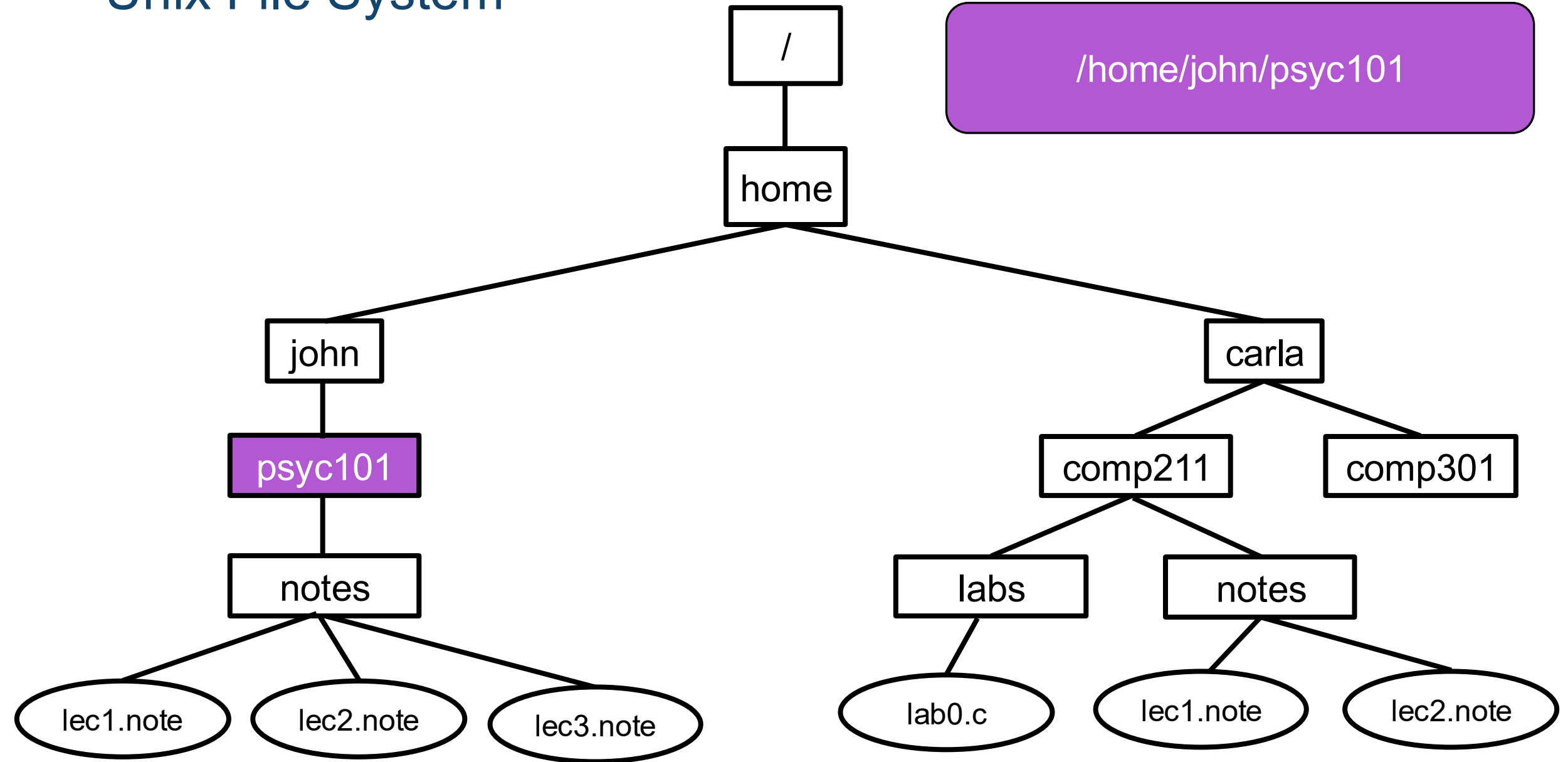


Unix File System

Every directory is identified by a unique pathname



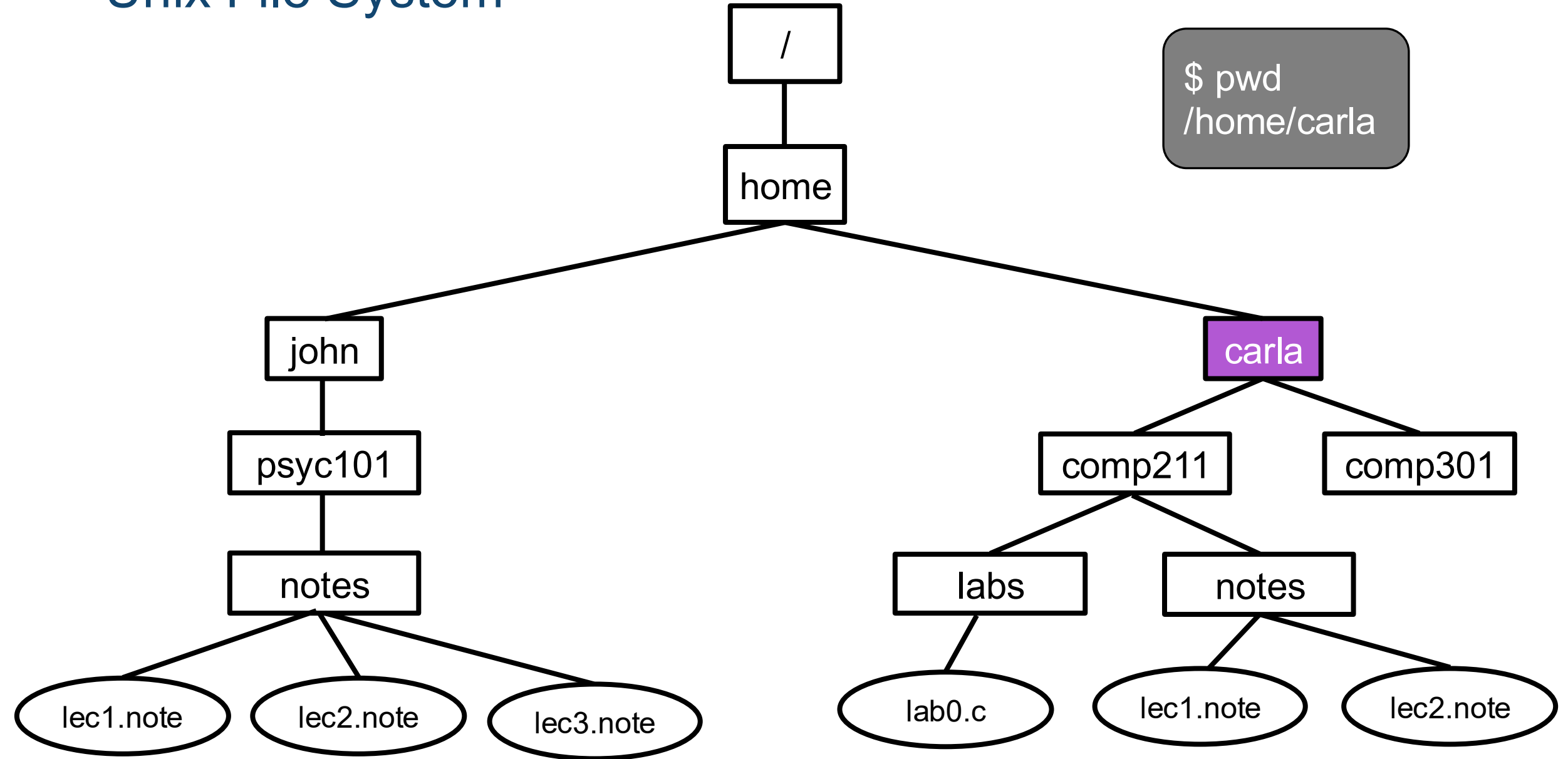
Unix File System



File System Commands

- **pwd** (print working directory)
 - Shows the directory you are currently in

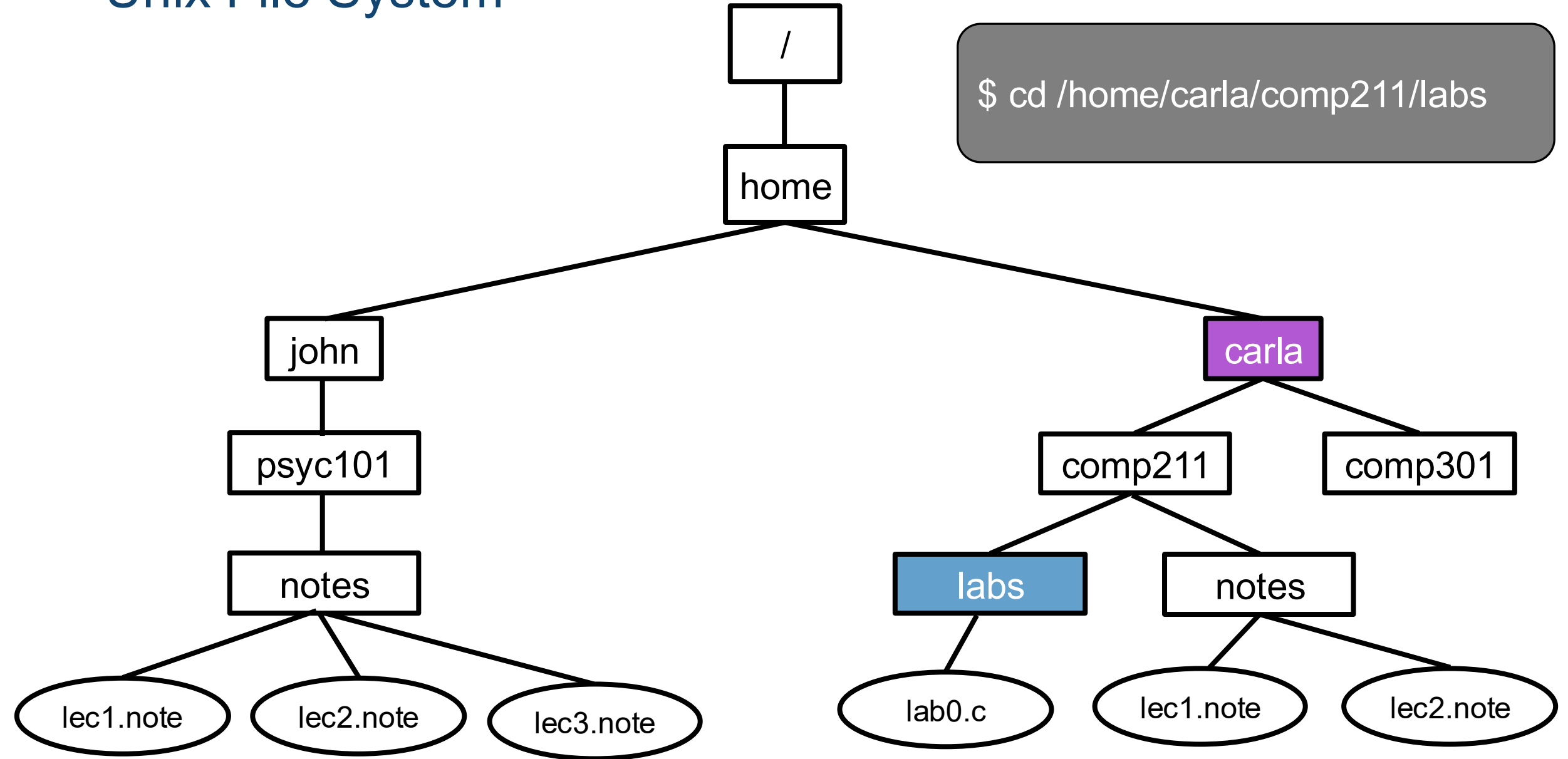
Unix File System



File System Commands

- **pwd** (print working directory)
 - Shows the directory you are currently in
- **cd** (change directory)
 - Changes the current working directory

Unix File System



Absolute vs Relative Paths

- **Absolute Path**

- A path that starts from the **root directory**
- Always unique

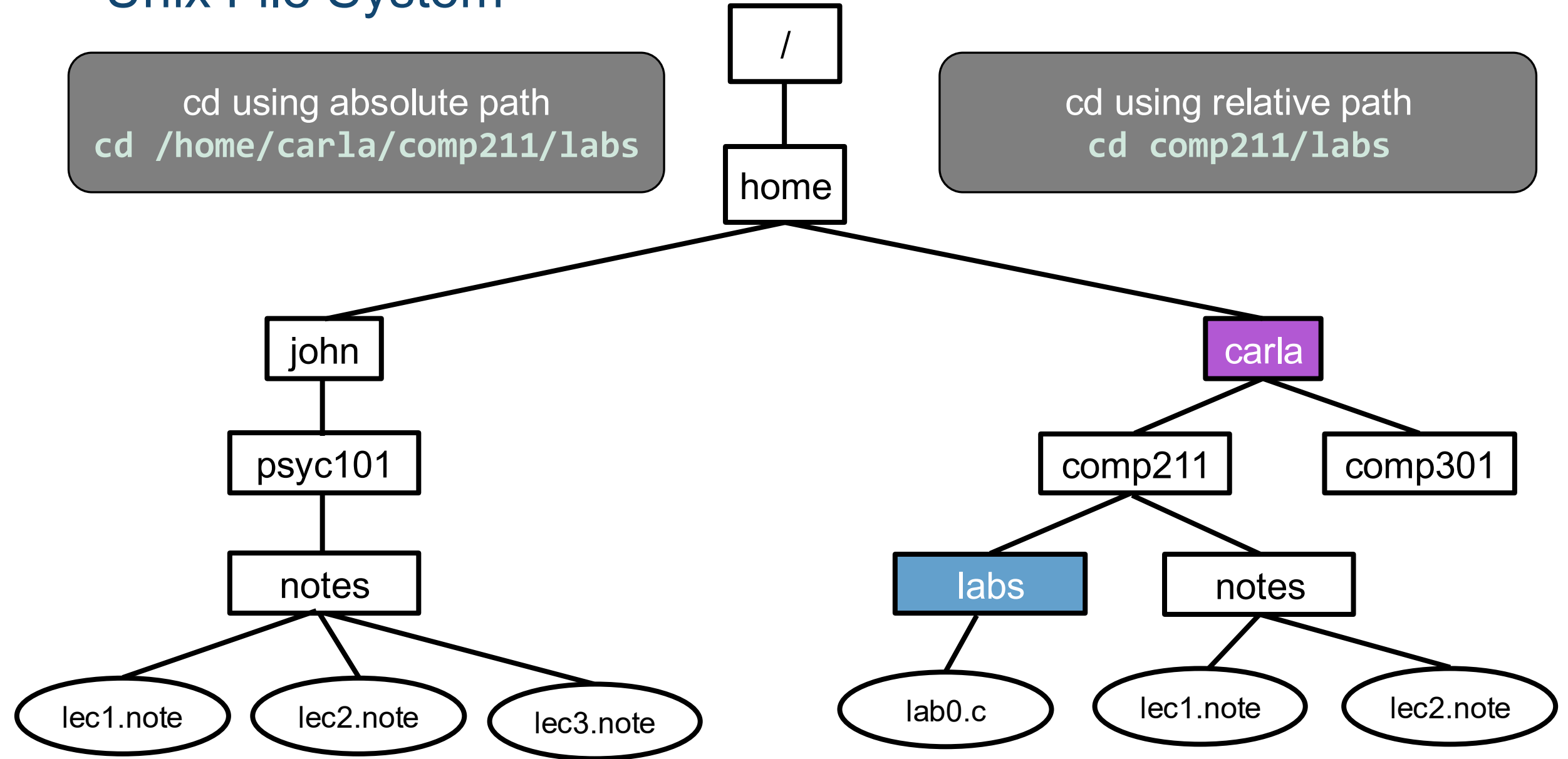
- **Relative Path**

- A path that starts from the **current working directory**
- **Parent directory:** `..` (two dots)
- **Current directory:** `.` (one dot)

Unix File System

cd using absolute path
`cd /home/carla/comp211/labs`

cd using relative path
`cd comp211/labs`



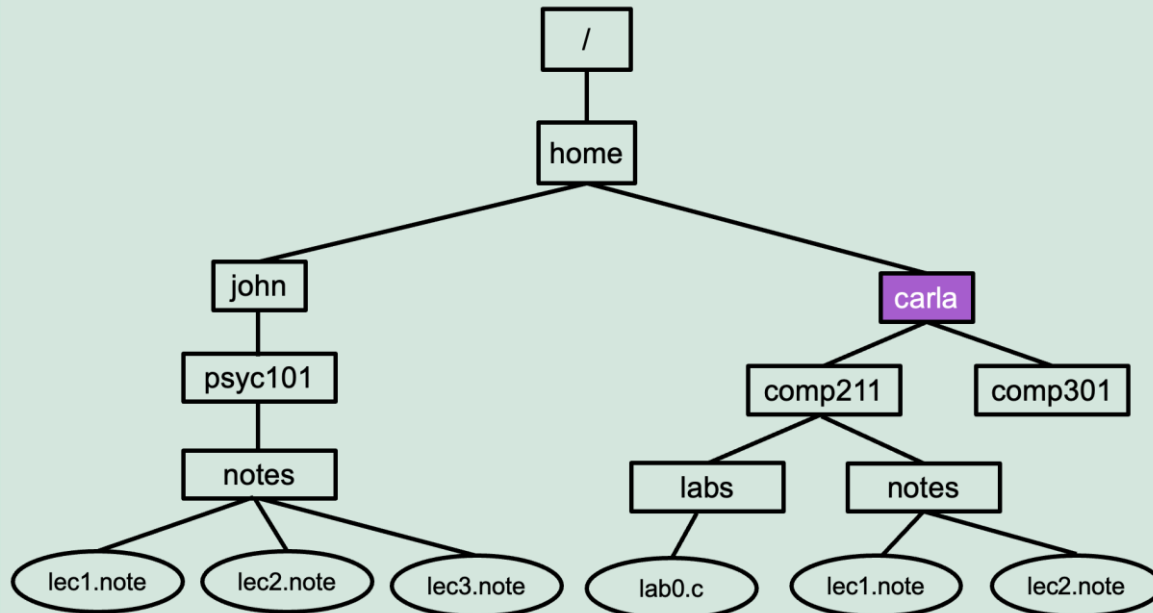
File System Commands

- **pwd** (print working directory)
 - Shows the directory you are currently in
- **cd** (change directory)
 - Changes the current working directory
- **mkdir** (make directory)
 - Creates a new directory
- **touch**
 - Creates a new empty file

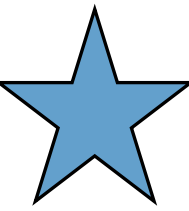
File System Commands

- If the current working directory is `carla` and we run the following commands, draw the new tree structure. Label the current working directory.

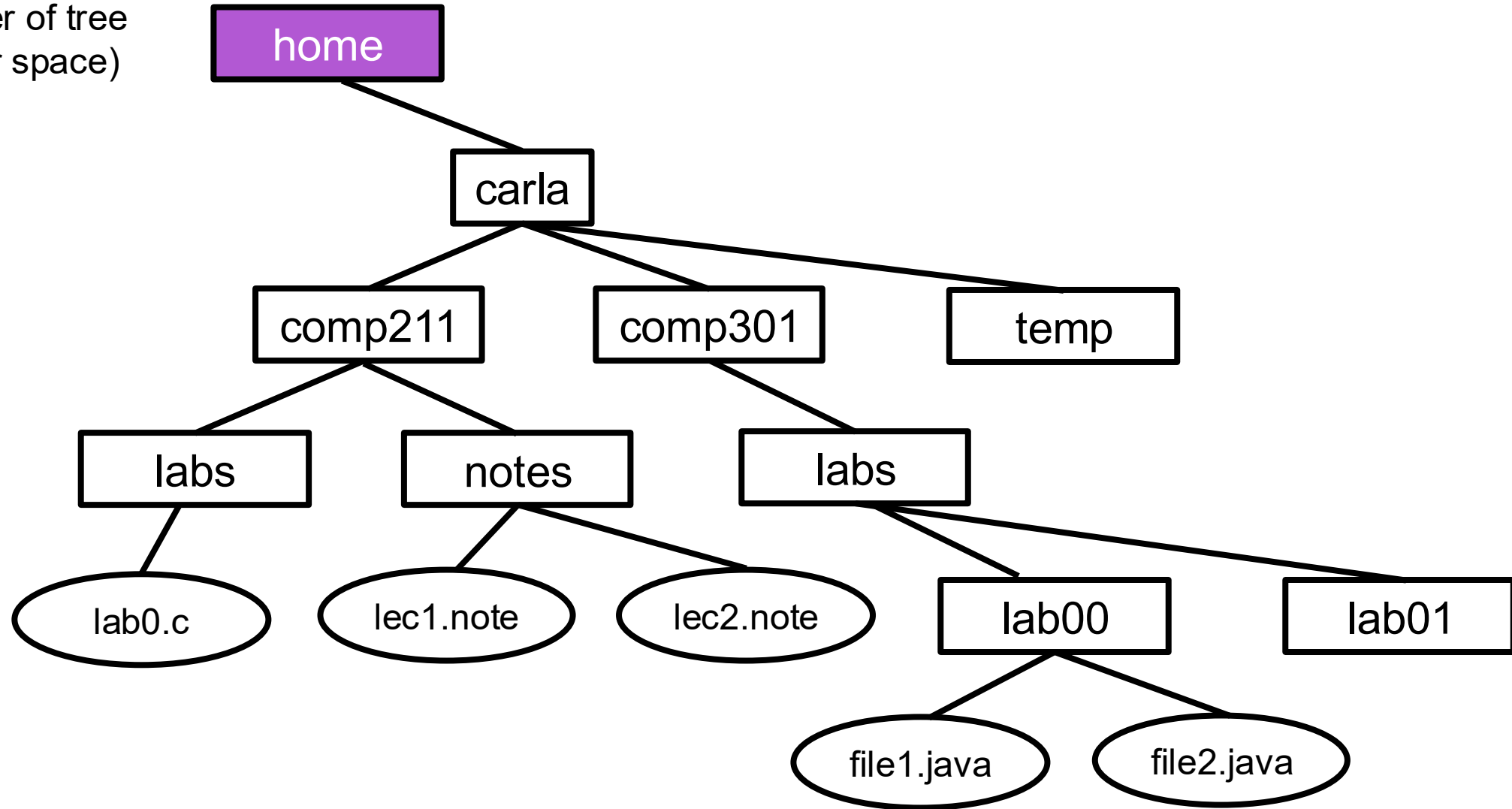
```
$ cd comp301
$ mkdir labs
$ mkdir labs/lab00
$ touch labs/lab00/file1.java
$ cd labs/lab00
$ touch file2.java
$ cd ..
$ mkdir lab01
$ cd ../../..
$ mkdir temp
$ cd ..
```



File System Commands



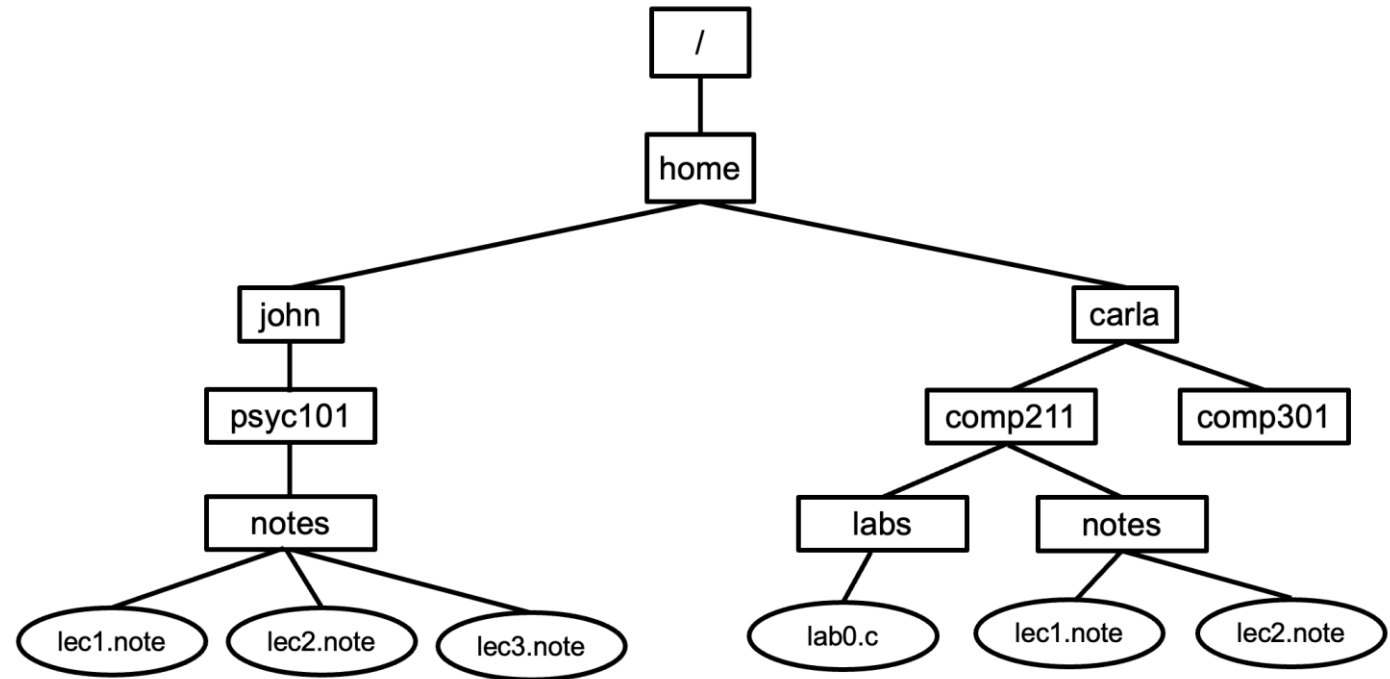
(remainder of tree
left out for space)



File System Tree

File System Tree

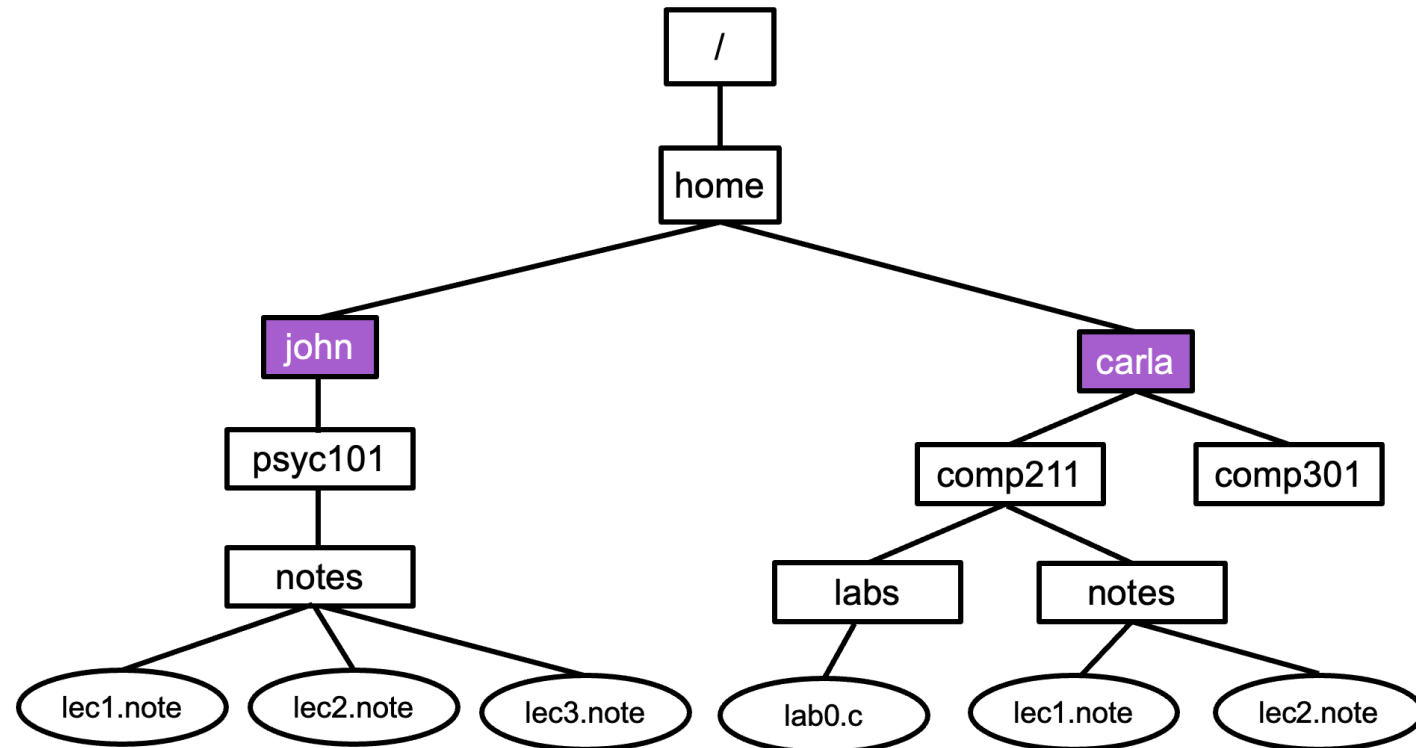
```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|           |-- lab0.c
|           |-- notes
|               |-- lec1.note
|               |-- lec2.note
|-- comp301
```



File System Tree

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|           |-- lab0.c
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|-- comp301
```

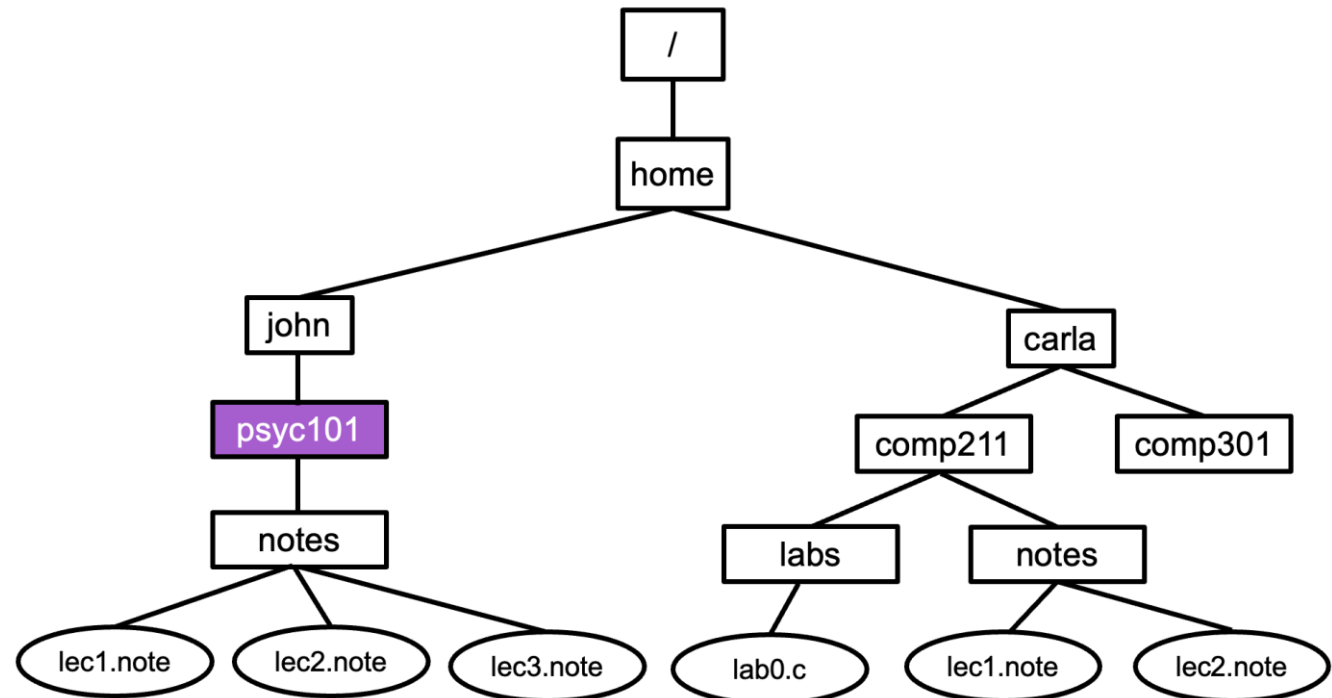
Within home, there are two subdirectories
– john and carla



File System Tree

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|           |-- lab0.c
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|-- comp301
```

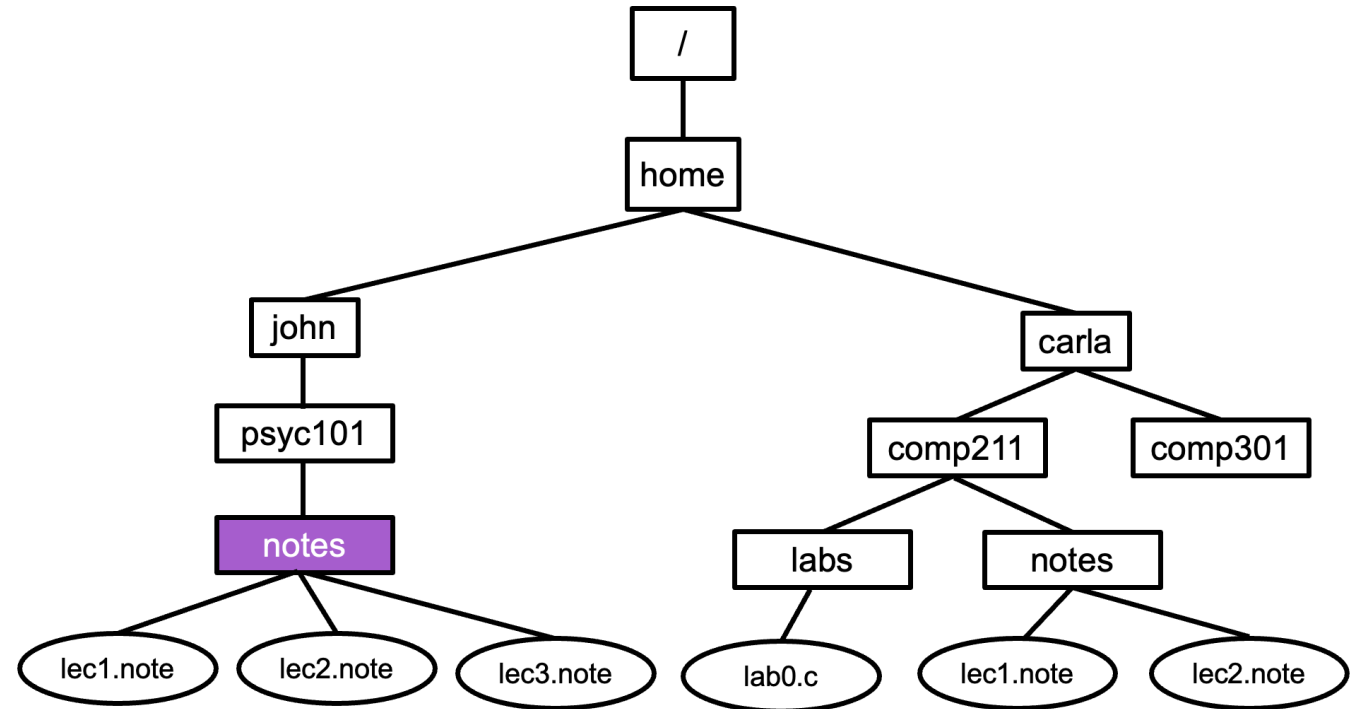
Within john, there is one subdirectory –
psyc101



File System Tree

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|       |-- lab0.c
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|-- comp301
```

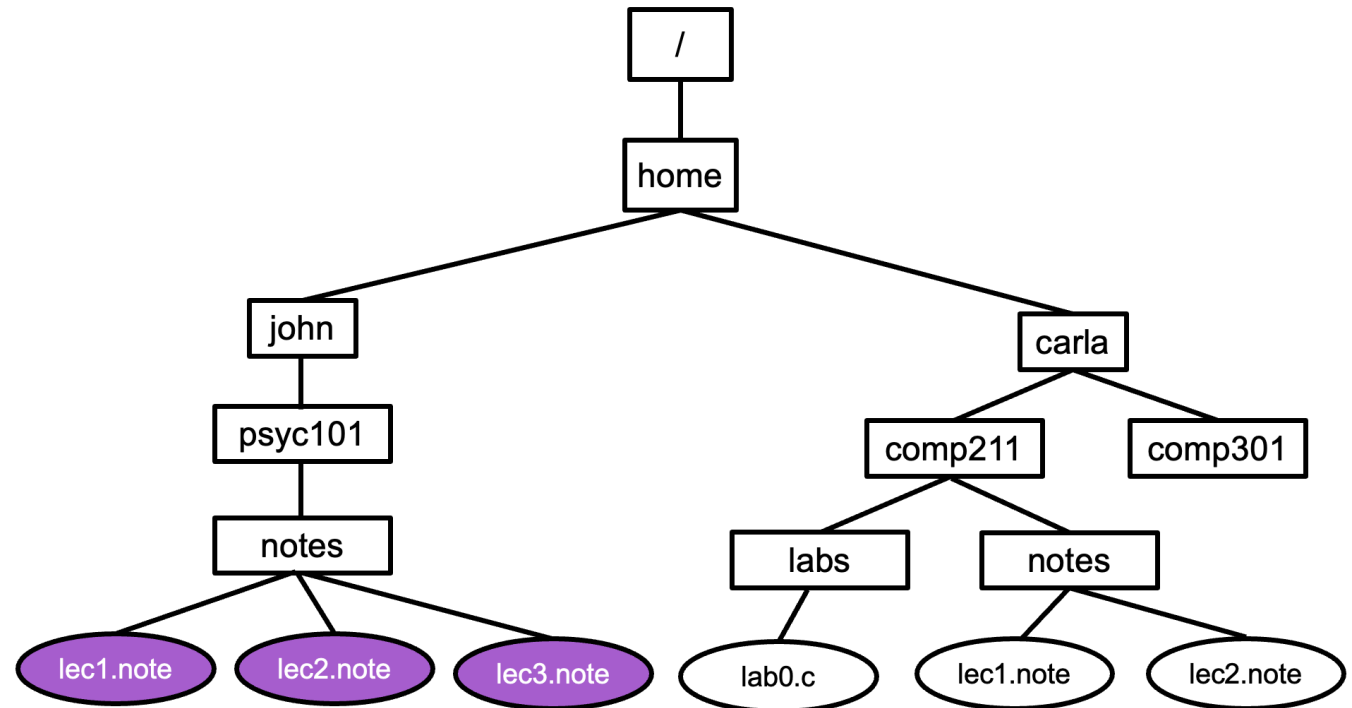
Within psyc101, there is one subdirectory
– notes



File System Tree

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|           |-- lab0.c
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|-- comp301
```

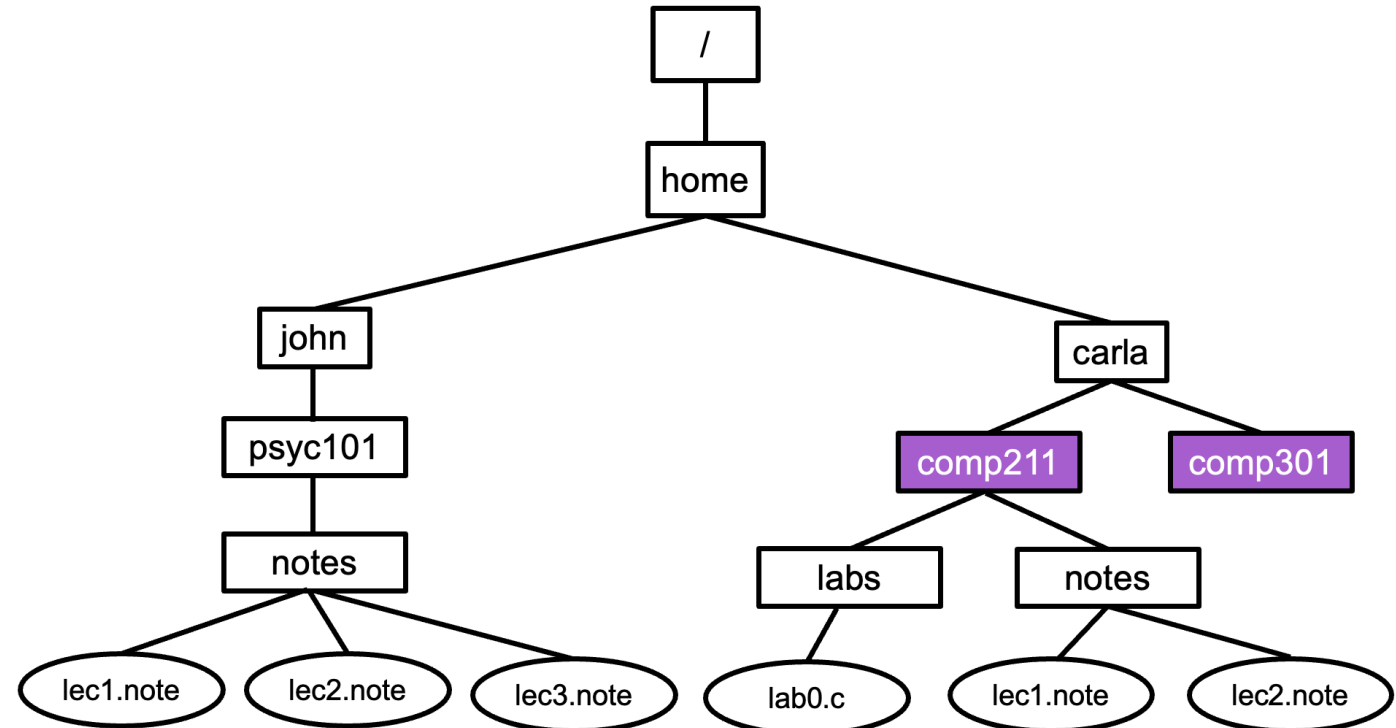
Within notes, there are three files –
lec1.note, lec2.note, and
lec3.note



File System Tree

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|           |-- lab0.c
|           |-- notes
|               |-- lec1.note
|               |-- lec2.note
|   |-- comp301
```

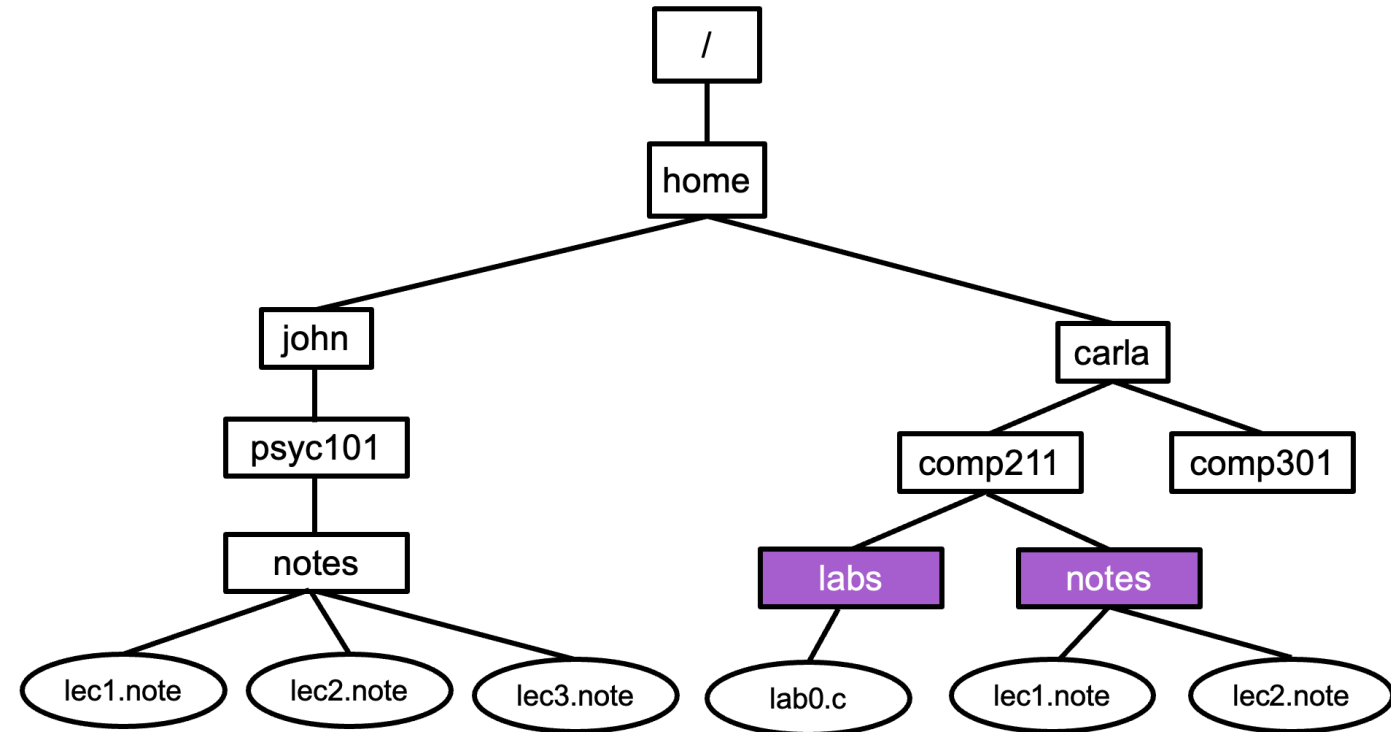
Within carla, there are two subdirectories
– comp211 and comp301



File System Tree

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|       |-- lab0.c
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|-- comp301
```

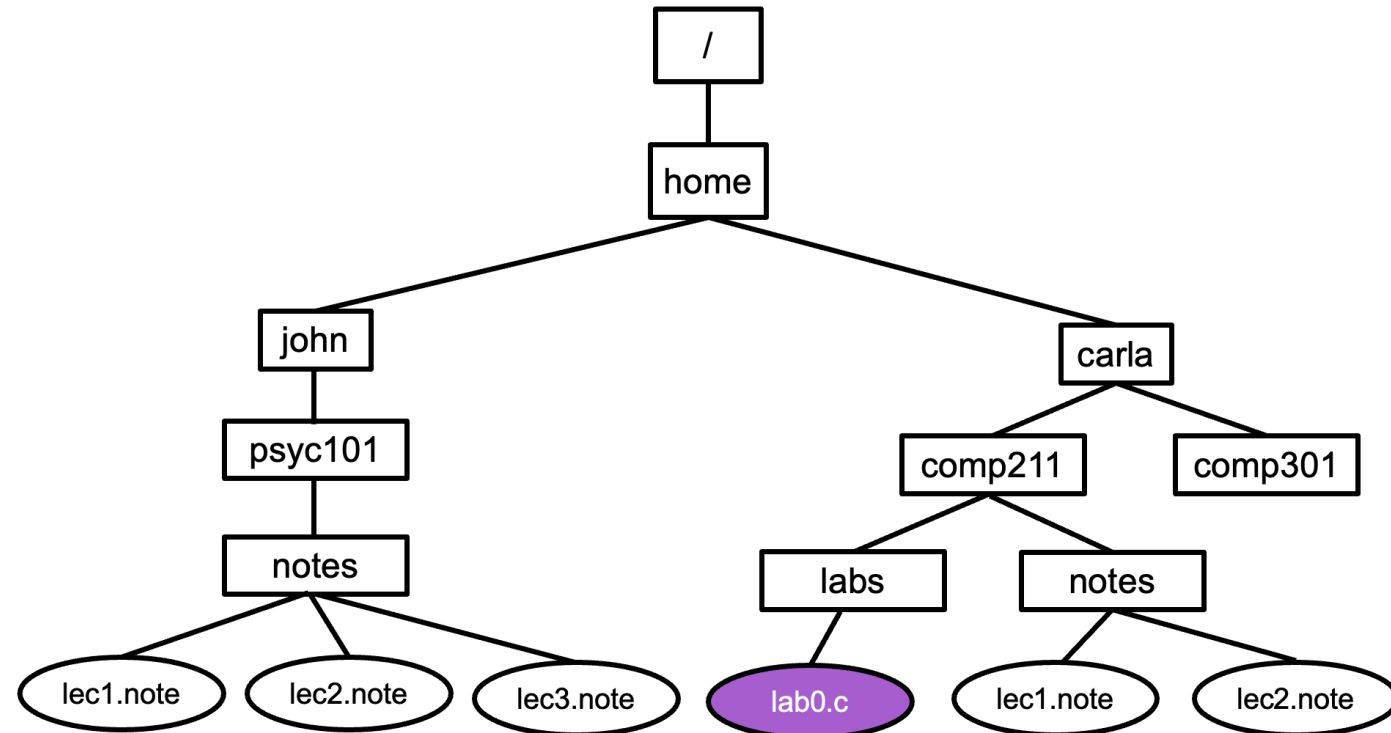
Within comp211, there are two subdirectories – labs and notes



File System Tree

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|           |-- lab0.c
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|-- comp301
```

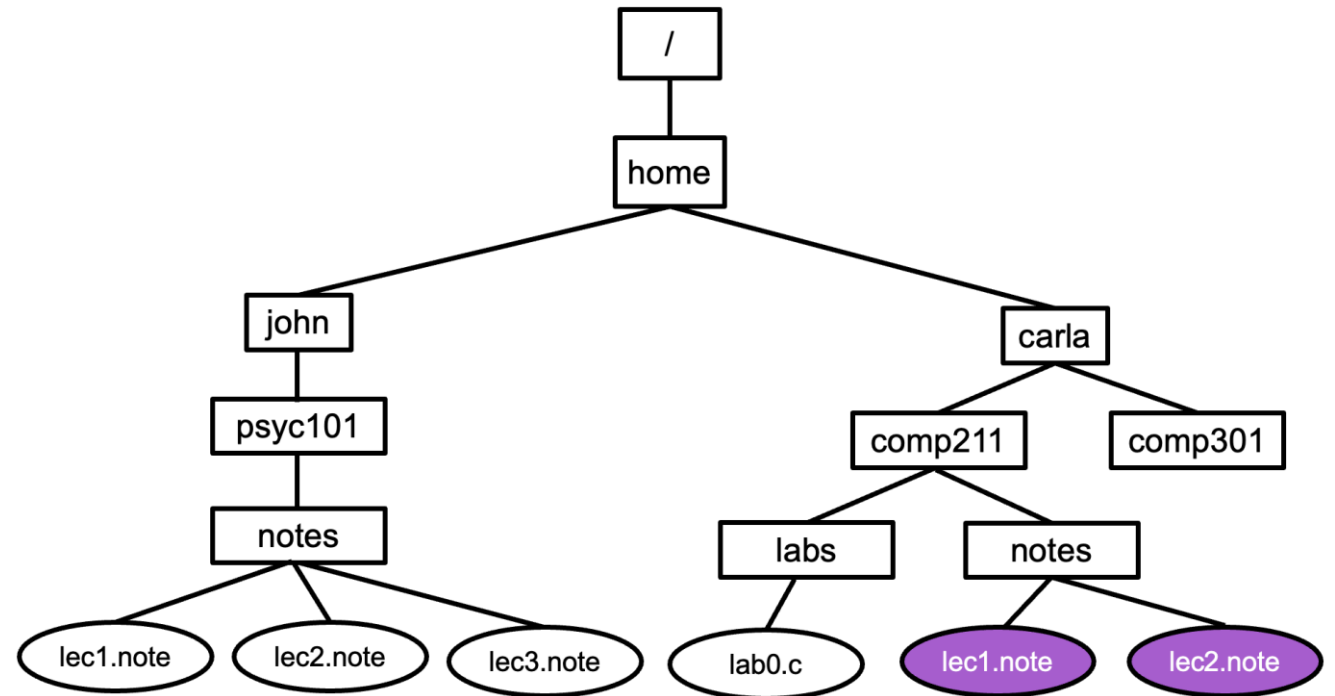
Within labs, there is one file – lab0.c



File System Tree

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|       |-- lab0.c
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|-- comp301
```

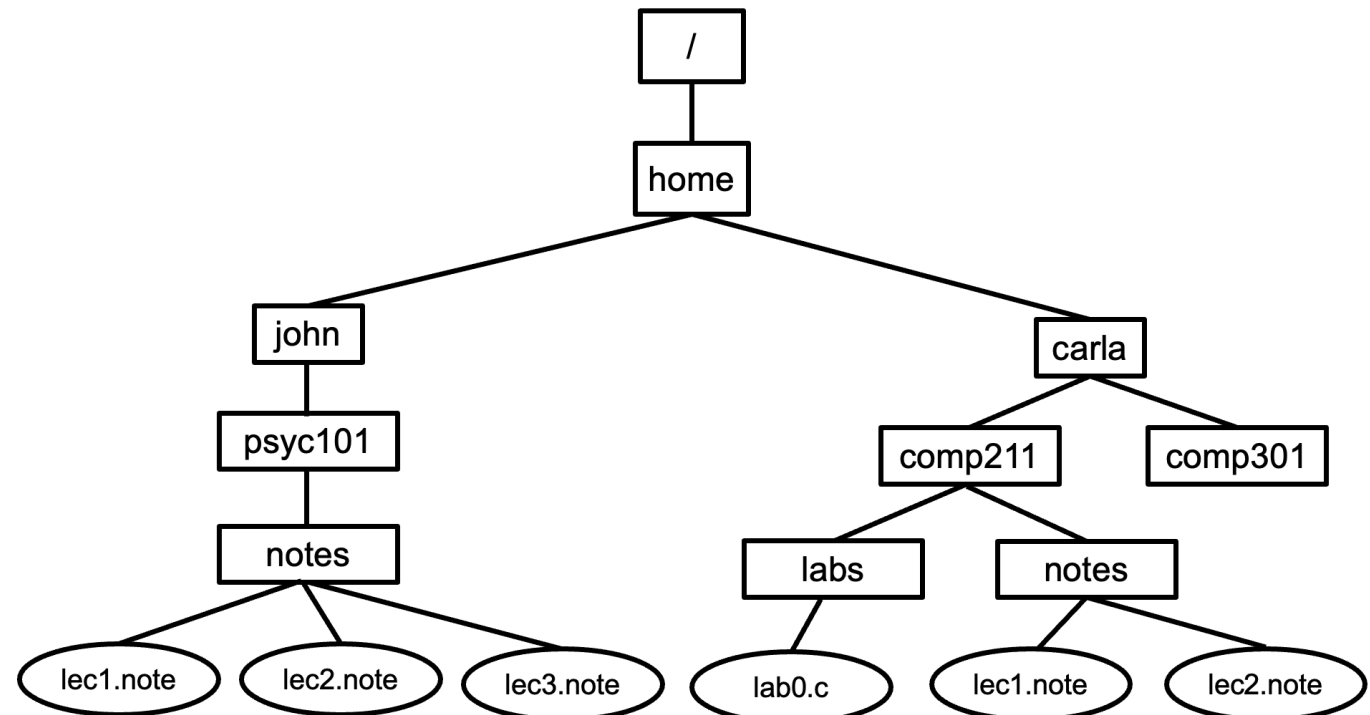
Within notes, there are two files –
lec1.note and lec2.note



File System Tree

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|           |-- lab0.c
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|-- comp301
```

Within comp301, there are no files or subdirectories



ls (list)

- Running **ls** alone lists the contents of the current working directory (cwd)

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
    |-- comp211
        |-- labs
        |   |-- lab0.c
        |   |-- notes
        |       |-- lec1.note
        |       |-- lec2.note
    |-- comp301
```

Shell prompt

```
$ pwd
/home/carla
$ ls
comp211  comp301
```

ls (list)

- Running **ls** with an argument lists the contents of the specified directory

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|           |-- lab0.c
|           |-- notes
|               |-- lec1.note
|               |-- lec2.note
|-- comp301
```

```
$ pwd
/home/carla
$ ls comp211/notes
lec1.note  lec2.note
```

Hidden Files and Directories

- A hidden file or directory is one whose name begins with a dot (.).
- Examples
 - `.git/`
 - `.vscode/`
 - `.bashrc`
- Purpose:
 - Store user/application configuration files
 - Keep clutter out of normal directory listings
- Visibility:
 - Normal `ls` won't show them
 - Use `ls -a` to display hidden files and directories

File System Commands

- **pwd** (print working directory)
 - Shows the directory you are currently in
- **cd** (change directory)
 - Changes the current working directory
- **mkdir** (make directory)
 - creates a new directory
- **touch**
 - creates a new empty file
- **ls** (list)
 - Lists the contents of the cwd
 - Can add an argument to list the contents of a specified directory
- **cp** (copy)
 - Copies a file or directory to another file or directory
- **mv** (move)
 - Moves a file or directory to another file or directory

cp (copy)

- Copy takes two arguments: The source and the destination

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
    |-- comp211
        |-- labs
        |   |-- lab0.c
        |   |-- notes
        |       |-- lec1.note
        |       |-- lec2.note
    |-- comp301
```

```
$ pwd
/home/carla/comp211/
$ cp notes/lec1.note ../comp301
```

The above command will
copy **lec1.note** to the
comp301 directory

cp (copy)

- Copy takes two arguments: The source and the destination

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|           |-- lab0.c
|           |-- notes
|               |-- lec1.note
|               |-- lec2.note
|-- comp301
    |-- lec1.note
```

```
$ pwd
/home/carla/comp211/
$ cp notes/lec1.note ../comp301
```

Now there are two copies of
lec1.note

The current working
directory is still **comp211**

cp (copy)

- Copy takes two arguments: The source and the destination

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|           |-- lab0.c
|           |-- notes
|               |-- lec1.note
|               |-- lec2.note
|-- comp301
    |-- lec1.note
```

```
$ pwd
/home/carla/comp211/
$ mv notes/lec2.note ../comp301
```

The above command will
move **lec2.note** to the
comp301 directory

cp (copy)

- Copy takes two arguments: The source and the destination

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|           |-- lab0.c
|           |-- notes
|               |-- lec1.note
|-- comp301
    |-- lec1.note
    |-- lec2.note
```

```
$ pwd
/home/carla/comp211/
$ mv notes/lec2.note ../comp301
```

lec2.note is no longer in
the **notes** directory.

The current working
directory is still **comp211**

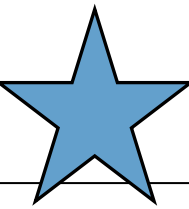
File System Commands

- Assume the directory structure to the right. Also assume the current working directory is `/Projects/Dev`. Write the command to perform the following tasks.
- Q1. Move `test_app.py` to the Code directory using relative paths for both the source and destination.
- Q2. Copy `readme.md` from the Docs directory to the Dev directory using absolute paths for both the source and destination.
- Q3. Rename `app.py` to `main.py` using a relative path for the source and an absolute path for the destination.

```
/Projects
|-- Dev
|   |-- Code
|       |-- app.py
|-- Docs
|   |-- readme.md
|-- Tests
|   |-- test_app.py
```



File System Commands



- Assume the directory structure to the right. Also assume the current working directory is `/Projects/Dev`. Write the command to perform the following tasks.
- Q1. Move `test_app.py` to the `Code` directory using relative paths for both the source and destination.
 - `mv ../Tests/test_app.py Code`
- Q2. Copy `readme.md` from the `Docs` directory to the `Dev` directory using absolute paths for both the source and destination.
 - `cp /Projects/Docs/readme.md /Projects/Dev`
- Q3. Rename `app.py` to `main.py` using a relative path for the source and an absolute path for the destination.
 - `mv Code/app.py /Projects/Dev/Code/main.py`

```
/Projects
|-- Dev
|   |-- Code
|       |-- app.py
|-- Docs
|   |-- readme.md
|-- Tests
|   |-- test_app.py
```

Removing Files

- The **rm** command is used to remove a file.
- **Be careful!!!** By default, this command permanently deletes files with no option for recovery.

```
$ pwd
/home/john/psyc101/notes
$ ls
lec1.note  lec2.note  lec3.note
$ rm lec1.note
$ ls
lec2.note  lec3.note
```

Removing Directories

- The `rmdir` command is used to remove a directory.
- By default, `rmdir` only removes empty directories
 - This feature prevents the user from accidentally removing content that they did not intend

```
$ pwd
/home/carla
$ ls
comp211    comp301
$ ls comp301
$ rmdir comp301
$ ls
comp211
```

Removing Directories

- The `rmdir` command is used to remove a directory.
- By default, `rmdir` only removes empty directories
 - This feature prevents the user from accidentally removing content that they did not intend

```
$ pwd
/home/carla/comp211
$ ls
labs    notes
$ ls labs
lab0.c
$ rmdir labs
rmdir: failed to remove labs: Directory not empty
```

Removing Directories

- A user can recursively remove the contents of a directory using `rm -r`, and can also forcefully remove files (get rid of all prompting before removing files) by adding the `-f` option too: `rm -rf`.

```
$ pwd
/home/carla/comp211
$ ls
labs      notes
$ ls labs
lab0.c
$ rm -rf labs
$ ls
notes
```